

CASE STUDY: REAL-TIME ANOMALY DETECTION USING VARIATIONAL AUTOENCODERS (VAES)

Introduction

Variational Autoencoders (VAEs) are a class of generative models used in machine learning and deep learning. They have found applications in a wide range of fields, including image generation, data compression, and natural language processing. In this case study, we will explore a real-time application of VAEs in the context of anomaly detection in industrial equipment.

Background

Imagine a manufacturing plant that produces various components using heavy machinery. The smooth operation of these machines is crucial for maintaining production efficiency and product quality. Any malfunction or anomaly in the machinery can lead to costly downtime and defects in the products. To prevent such issues, real-time anomaly detection is essential.

The Problem

The manufacturing plant has various sensors that continuously monitor the machinery's performance, collecting data such as temperature, pressure, vibration, and more. The challenge is to detect anomalies in this data as soon as they occur to minimize downtime and maintenance costs.

Solution using VAEs

Variational Autoencoders offer an effective solution for real-time anomaly detection in this scenario. Here's how it works:

1. **Data Collection:** Data from sensors is collected over time, forming a time series dataset. This dataset contains both normal operating conditions and instances with anomalies.
2. **Data Preprocessing:** The data is preprocessed to normalize and clean it, ensuring it is suitable for training the VAE.
3. **VAE Training:** A VAE model is trained using preprocessed data. The VAE is trained to learn a compressed representation of the data, effectively encoding the normal patterns in the machinery's behavior.

Sample VAE Implementation

Code:

```
import numpy as np
import pandas as pd
data = pd.read_csv('./anamoly_data.csv')
data.head(5)
```

Output:

	Timestamp	Temperature	Pressure	Vibration
0	2023-01-01 00:00:00	25.000000	1100.000000	0.000000
1	2023-01-01 00:01:00	25.021817	1099.996192	0.020906
2	2023-01-01 00:02:00	25.043633	1099.984770	0.041582
3	2023-01-01 00:03:00	25.065448	1099.965732	0.061803
4	2023-01-01 00:04:00	25.087262	1099.939083	0.081347

Explanation:

Data from sensors collected over time is loaded from the csv file 'anamoly_data.csv'.

Code:

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, Lambda
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import mse
# ---- DATA PREPARATION ----
sensor_data = np.column_stack((data['Temperature'].values,
data['Pressure'].values, data['Vibration'].values))
```

Explanation:

The code imports necessary modules from TensorFlow and Keras for building neural networks. It then combines the 'Temperature', 'Pressure', and 'Vibration' columns from the data DataFrame into a 2D array called sensor_data.

Code:

```
# ---- VAE MODEL ----
latent_dim = 2
input_dim = 3
encoder_inputs = Input(shape=(input_dim,))
x = Dense(128, activation='relu')(encoder_inputs)
z_mean = Dense(latent_dim)(x)
z_log_var = Dense(latent_dim)(x)
def sampling(args):
    z_mean, z_log_var = args
    epsilon =
tf.keras.backend.random_normal(shape=(tf.shape(z_mean)[0],
latent_dim))
    return z_mean + tf.exp(0.5 * z_log_var) * epsilon

z = Lambda(sampling)([z_mean, z_log_var])
decoder_inputs = Input(shape=(latent_dim,))
x = Dense(128, activation='relu')(decoder_inputs)
outputs = Dense(input_dim, activation='linear')(x)

decoder = Model(decoder_inputs, outputs)
vae = Model(encoder_inputs, decoder(z))
```

Explanation:

The code establishes a Variational Autoencoder (VAE) that compresses 3D input data into a 2D latent space and then decodes it back, using layers to define the encoder and decoder processes.

In a VAE, the sampling step introduces randomness, ensuring that the model doesn't simply learn a deterministic mapping between inputs and the latent space. Instead, for each input, it learns a distribution in the latent space. The `z_mean` and `z_log_var` vectors represent the parameters of this distribution. Sampling allows the VAE to generate varied outputs and aids in the model's generalization.

Code:

```
class VAE LossLayer(tf.keras.layers.Layer):
    def __init__(self, **kwargs):
        super(VAE LossLayer, self).__init__(**kwargs)

    def call(self, inputs):
        x, x_decoded_mean, z_log_var, z_mean = inputs
        xent_loss = tf.reduce_mean(mse(x, x_decoded_mean))
        kl_term = 1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)
        kl_loss = -0.5 * tf.reduce_mean(kl_term)
        loss = xent_loss + kl_loss
        self.add_loss(loss)
        return x # Can return anything, doesn't matter as this layer is just
for adding the loss
```

Explanation:

The code defines a custom Keras layer, VAE LossLayer, which calculates the loss for a Variational Autoencoder (VAE) by combining the reconstruction loss and the KL divergence. This loss ensures the VAE learns meaningful encodings in the latent space. The VAE model then uses this custom layer to compute its overall loss and is compiled with the Adam optimizer for training.

Code:

```
outputs = decoder(z)
vae_outputs = VAE LossLayer()([encoder_inputs, outputs, z_log_var,
z_mean])
vae = Model(encoder_inputs, vae_outputs)
vae.compile(optimizer=Adam())
```

Explanation:

The code decodes latent representations into outputs, calculates the model's error using a special layer VAE LossLayer, builds the full model from input to output, and then readies it for training with the Adam method.

Code:

```
# ---- TRAIN VAE ----
train_size = int(0.8 * len(sensor_data))
x_train = sensor_data[:train_size]
x_valid = sensor_data[train_size:]
epochs = 50
batch_size = 32
vae.fit(x_train, x_train,
        shuffle=True,
        epochs=epochs,
        batch_size=batch_size,
        validation_data=(x_valid, x_valid))
```

Output:

```
Epoch 1/50
108/108 [=====] - 1s 5ms/step
- loss: 253493.5781 - val_loss: 84043.8594
Epoch 2/50
108/108 [=====] - 0s 4ms/step
- loss: 44589.9336 - val_loss: 27646.7246
...
108/108 [=====] - 1s 5ms/step
- loss: 257.5038 - val_loss: 362.1933
108/108 [=====] - 0s 2ms/step
136/136 [=====] - 0s 2ms/step
```

Explanation:

In this code, data is divided into training and validation sets. This lets the model learn from one part and tests its performance on unseen data, preventing overfitting. The model trains by reviewing the entire dataset multiple times, as specified by epochs, and processes data in groups determined by batch_size, which affects both the speed and quality of training. Training with vae.fit(): The model tweaks its settings to better



TEKNOTURF

predict the training data, while also checking its accuracy on the validation data.

Code:

```
# ---- ANOMALY DETECTION ----
reconstructed_train = vae.predict(x_train)
train_error = np.mean(np.square(x_train - reconstructed_train), axis=-1)

threshold = np.percentile(train_error, 99)

reconstructed_data = vae.predict(sensor_data)
reconstruction_error = np.mean(np.square(sensor_data -
reconstructed_data), axis=-1)

anomalies = reconstruction_error > threshold
print("No. of anomalies in the given data: {} out of
{}".format(np.sum(anomalies), len(sensor_data)))
```

Output:

No. of anomalies in the given data: 47 out of 4321

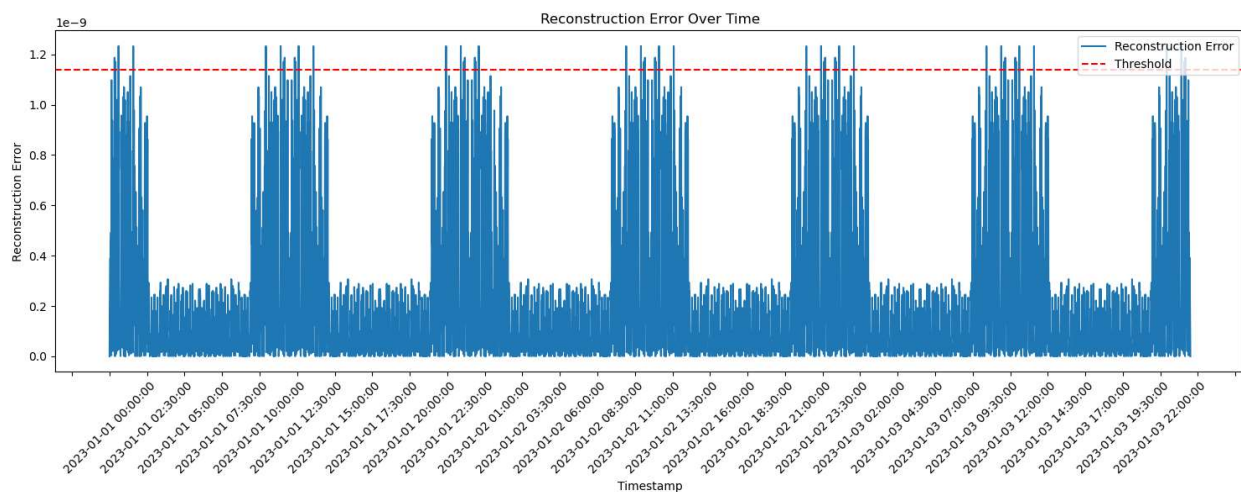
Explanation:

The code uses the trained model to rebuild the training data and calculate the error between original and rebuilt values. It sets a threshold based on the 99th percentile of this error. Then, it rebuilds the entire dataset and identifies data points with errors exceeding the threshold as anomalies, finally printing the number of anomalies detected.

Code:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(15, 6))
plt.plot(data['Timestamp'], reconstruction_error, label='Reconstruction Error')
plt.axhline(y=threshold, color='r', linestyle='--', label='Threshold')
plt.title("Reconstruction Error Over Time")
plt.legend()
plt.xlabel("Timestamp")
plt.ylabel("Reconstruction Error")
# Rotate the x-axis labels for better readability and set an interval for them
plt.xticks(rotation=45)
plt.gca().xaxis.set_major_locator(plt.MaxNLocator(nbins=40))
plt.tight_layout() # Adjust layout to ensure labels fit
plt.show()
```

Output:



Explanation:

The chart visualizes the error between the original and rebuilt data over time. A red dashed line indicates a set threshold. Any spike in the error above this line suggests a potential anomaly at that time.

Challenges in Implementation

Implementing real-time anomaly detection using VAEs in an industrial setting poses several challenges:

1. **Data Quality:** Ensuring the quality and reliability of sensor data is critical. Noisy or incomplete data can lead to false alarms or missed anomalies.
2. **Model Training:** Training for a VAE requires a large amount of labeled data, which can be expensive and time-consuming to collect in an industrial environment.
3. **Latency:** Achieving real-time detection with minimal latency is crucial. The VAE model must be efficient enough to handle data streams in real-time.
4. **Threshold Setting:** Choosing an appropriate threshold for anomaly detection can be challenging. It requires a balance between false positives and false negatives.
5. **Adaptability:** The machinery's behavior may change over time due to maintenance or process adjustments, requiring the model to adapt continuously.
6. **Interpretability:** Understanding the VAE's latent representations and reconstruction errors can be complex, making it challenging to diagnose the root causes of anomalies.